WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Work Package · Backend C

Backend | NestJS + Prisma | Transactions

PROJECT	TEAM LEADER	OWNER
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Rami Al Shaar**

ROLE	AREA
Backend developer	**Attempt state · Timing · Scoring · Start · MCQ · Run / Submit / Finish**

Everything that happens while a student takes a test. This is the part the project is graded on most.

Use the class pattern: the same NestJS layout you built in B4F Board: module, controller, service, DTOs, and `PrismaService` injected into the service (as in the Session 21 cutover). Controllers only pass data; services hold the rules; database access only in services; format checks only in DTOs.

Full reference: this package is a summary of `docs/TASKS.md`. Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md`. If this package and the docs ever disagree, the docs win; tell Ali.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Attempt state (first PR, others depend on it)	`attempt-state/attempt-state.service.ts`
T2	Scoring service	`attempt-state/scoring.service.ts`
T3	Student tests (list, one test, lock rules)	`student/student-tests.service.ts`, `student-tests.controller.ts`
T4	Start or resume an attempt (pledge, order, session)	`student/attempts.service.ts`, `dto/start-attempt.dto.ts`
T5	Student view mapper + get attempt	`student/mappers/student-view.mapper.ts`, `student-attempts.controller.ts`
T6	MCQ autosave	`student/mcq.service.ts`, `dto/mcq-answer.dto.ts`
T7	MCQ submit	`student/mcq.service.ts`, `dto/mcq-submit.dto.ts`
T8	Coding Run	`student/coding.service.ts`, `dto/code.dto.ts`
T9	Coding Submit	`student/coding.service.ts`
T10	Coding Finish	`student/coding.service.ts`

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere
Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone
Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration
Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Your Tasks, Step by Step

T1 Attempt state (first PR, others depend on it)

`api/src/attempt-state/attempt-state.service.ts` · `attempt-state.module.ts`

1. Write the attempt state table (every state, every request, every result) as a comment at the top of `attempt-state.service.ts` (Phase 0).
2. `isOverdue(attempt, now)`: `status === IN_PROGRESS && now > deadline + GRACE_SECONDS`.
3. `expireOverdue()`: one `updateMany`: every overdue attempt → `EXPIRED`, `score: null`. Logs how many. Touches only `IN_PROGRESS` attempts.
4. `assertAcceptingRequests(attempt)`: `SUBMITTED` → `409 This test has already been submitted.`; `TERMINATED` → `409 This attempt was ended because the exam rules were broken.`; `EXPIRED` or overdue → store `EXPIRED`, log `late request rejected`, `409 Time is over. This attempt expired and was not submitted.`
5. `hasAttempts(testId)`: any attempt for this test. `hasActiveAttempts(testId)`: any `IN_PROGRESS` attempt (after `expireOverdue()`).
6. Test with a fake `ClockService` (or a short test duration): 1 second before deadline, exactly the deadline, deadline + 9 s, deadline + 11 s.

✔ Done when: requests up to deadline + 10 s are accepted; later ones are rejected and the attempt is stored as `EXPIRED` with `score: null`; an overdue attempt shows `EXPIRED` on the next read.

T2 Scoring service

`api/src/attempt-state/scoring.service.ts`

Used by MCQ submit, Finish, and Esmaeel's termination.

1. `scoreMcqFromSaved(tx, attemptId)`: correct saved answers ÷ all questions × 100, `round1`.
2. `scoreCoding(tx, attemptId)`: unsubmitted problems → `points: 0`; sum of points ÷ sum of problem points × 100, `round1`.
3. `CodingService.finish` uses `scoreCoding`, so Finish and termination always give the same number.

✔ Done when: `ScoringService` gives the same score for Finish and for a termination of the same data.

T3 Student tests (list, one test, lock rules)

`student/student-tests.service.ts` · `student/student-tests.controller.ts`

1. `listForStudent(userId)`: `expireOverdue()`, then published tests in order with `state`, `lockReason`, `attemptId`, `score` (rules in section 5 of this package).
2. `getForStudent(userId, testId)`: `404 Test not found.` / `403 This test is not available.` Log forbidden access.
3. `assertCanStart(userId, test)`: R-2 → `403 Complete the <previous title> first.` Log forbidden access.
4. "Finished" means `SUBMITTED`, `EXPIRED` or `TERMINATED`: a student whose time ran out or who was terminated is never blocked from the next test.
5. `score` is set for `SUBMITTED` and `TERMINATED`; `null` for `EXPIRED`.

✔ Done when: students see only published tests, in order, with the right state and lock reason; unpublished → `403`; test 2 before test 1 → `403` with the reason.

T4 Start or resume an attempt

`student/attempts.service.ts` · `student/dto/start-attempt.dto.ts`

1. `StartAttemptDto { acceptedRules }` with `@Equals(true)` → `400 You must accept the exam rules to start.`
2. Checks in order: test exists (`404`), published (`403`), an `IN_PROGRESS` not-overdue attempt exists → return it (`200`), already finished → `409 You have already finished this test.`, previous test not finished → `403` with the reason.
3. New attempt in **one transaction**: `startedAt = ClockService.now()`, `deadline = startedAt + durationMinutes`, `buildAttemptOrder` (Majed) for MCQ tests stored in `questionOrder` / `optionOrder`, `ProblemResult` rows (run count 0) for coding, and `integrityService.issueSessionKey(tx, attemptId, false)` (Esmaeel). `201`.
4. Resume (`IN_PROGRESS`): same attempt and deadline, `issueSessionKey(tx, attemptId, true)`. `200`.
5. Catch `P2002` (two clicks) and return the existing attempt.
6. Log `attempt started`.

✔ Done when: starting again returns the same attempt with the same deadline; two fast clicks create one attempt; start needs the pledge.

T5 Student view mapper + get attempt

`student/mappers/student-view.mapper.ts` · `student/student-attempts.controller.ts`

1. `getForStudent(userId, attemptId)`: own attempts only (`404 Attempt not found.` otherwise), `expireOverdue()` first, shaped by the mapper.
2. The mapper applies the stored order (`applyQuestionOrder`, `applyOptionOrder`) and sets `selectedOptionId` and `integrity` (`integrityService.getIntegrity`). `sessionKey` only in the start response.
3. Use Prisma `select` (not `include`) for options and test cases, so `isCorrect` and hidden cases are never loaded.
4. `@UseGuards(JwtAuthGuard, RolesGuard, AttemptSessionGuard)` on `StudentAttemptsController`.

```
// student/mappers/student-view.mapper.ts  (the ONLY place that builds student attempt shapes)
toStudentQuestion(question, savedOptionId) → { id, text, options: [{ id, text }], selectedOptionId }   //
never isCorrect
                                // questions and options are put in the attempt's stored order first
toStudentProblem(problem, result) → { id, title, description, functionName, points,
                                      sample: { args, expected }, runCount, runsRemaining, submission }
                                      // only the SAMPLE case, never HIDDEN
```

✔ Done when: no student response contains `isCorrect` or hidden cases (checked in Swagger and the Network tab); the shuffled order stays the same after a refresh.

T6 MCQ autosave — `PUT /student/attempts/:attemptId/mcq-answers/:questionId`

`student/mcq.service.ts` · `student/dto/mcq-answer.dto.ts`

1. `McqService.saveAnswer(userId, attemptId, questionId, dto)`, checks in the order of the table below.
2. `upsert` on `(attemptId, questionId)` with `isCorrect` calculated on the server, **never returned**.
3. Response: `{ questionId, optionId, savedAt }`.

#	Problem	Status	Message
1	`optionId` is not a positive whole number or `null`	400	`optionId must be a positive whole number or null.`
2	Attempt not found / not own	404	`Attempt not found.`
3	Wrong or missing attempt session	409	(guard, `API.md` 19.4)
4	Attempt is not an MCQ attempt	400	`This action is only for MCQ tests.`
5	Attempt finished, or now > deadline + 10 s	409	(same messages as submit)
6	Question not in this test	400	`An answer refers to a question that is not part of this test.`
7	Option not in this question	400	`An answer refers to an option that does not belong to its question.`

✔ Done when: autosaved choices come back after a refresh.

T7 MCQ submit — `POST /student/attempts/:attemptId/mcq-submit`

`student/mcq.service.ts` · `student/dto/mcq-submit.dto.ts`

1. Load the attempt with its questions and options.
2. `assertAcceptingRequests(attempt)`.
3. Validate answers: malformed → `400 Answers must be a list of { questionId, optionId }.`; duplicate → `400 Each question can be answered only once.`; unknown question / option of another question → the messages from T6.
4. In **one transaction**: conditional `updateMany` on `status: IN_PROGRESS` (0 rows → `409`), delete the saved answers and create the `McqAnswer` rows (unanswered questions too, `optionId: null`, `isCorrect: false`), save `score = round1(correct / total × 100)`, `status: SUBMITTED`, `submittedAt: now`.
5. Log `MCQ submitted`. Response: `16 / 20 correct (80%)` shape from `API.md` section 10.
6. Check by hand: 16 of 20 → `80`; 1 of 3 → `33.3`; 0 answers → `0` (a real 0, status SUBMITTED).

✔ Done when: MCQ scores match a hand calculation; double submit → `409`.

T8 Coding Run

`student/coding.service.ts` · `student/dto/code.dto.ts`

Common checks for Run, Submit and Finish (`API.md` section 11): empty code → `400 Code cannot be empty.`; over 10,000 characters → `400 Code cannot be longer than 10,000 characters.` (in `CodeDto`); attempt not found → `404`; not a coding attempt → `400 This action is only for the coding test.`; problem not in this test → `404 Problem not found in this test.`; then the state checks.

1. `assertAcceptingRequests`.
2. Reserve a run: conditional `updateMany` with `runCount < MAX_RUNS_PER_PROBLEM`. 0 rows → `409 You have used all 20 runs for this problem.`
3. `codeExecutionService.run(problem, code)`.
4. If it throws `503`: give the run back (`decrement: 1`) and rethrow.
5. Return `RunResult` = outcome + `runCount` + `runsRemaining`. No score, nothing else stored.

✔ Done when: Run uses the sample case, never stores a score, and the 21st run is rejected; a `503` does not use a run.

T9 Coding Submit

`student/coding.service.ts`

1. `assertAcceptingRequests`. Already submitted → `409 This problem has already been submitted.`
2. `codeExecutionService.submit(problem, code)`. `503` → rethrow, nothing saved.
3. `points = round1(problem.points × passedCount / totalCount)`.
4. Conditional `updateMany` where `submittedAt: null` (0 rows → `409`). Save code, counts, points, `submittedAt`.
5. Log `problem submitted`. Return `SubmitResult` (counts only: `4 of 5 tests passed`).

✔ Done when: Submit works once per problem and stores counts and points.

T10 Coding Finish

`student/coding.service.ts`

1. `assertAcceptingRequests`.
2. In **one transaction**: conditional `updateMany` on `IN_PROGRESS`; `scoringService.scoreCoding` (sets `points: 0` on unsubmitted problems); `status: SUBMITTED`; `submittedAt: now`.
3. Log `coding finished`. Return the finish result.

✔ Done when: Finish runs in one transaction; unsubmitted problems score 0.

4 States, Timing and Shapes

```
state for a student card (first match wins):
the student has an attempt:
   IN_PROGRESS (and not overdue)   → IN_PROGRESS
   SUBMITTED                       → SUBMITTED
   EXPIRED (or overdue)            → EXPIRED
   TERMINATED                      → TERMINATED
no attempt:
   test order is 1                 → AVAILABLE
   previous test is finished       → AVAILABLE
   otherwise                       → LOCKED, lockReason "Complete the <previous title> first."

GRACE_SECONDS = 10
startedAt     = server time when the attempt is created
deadline      = startedAt + durationMinutes
overdue       = status is IN_PROGRESS AND now > deadline + GRACE_SECONDS
"finished"    = SUBMITTED, EXPIRED or TERMINATED
```

Used by	Name you provide
Majed, Esmaeel	`AttemptStateService.expireOverdue()`, `hasAttempts(testId)`, `hasActiveAttempts(testId)`, `assertAcceptingRequests(attempt)`
Esmaeel	`ScoringService.scoreMcqFromSaved(tx, attemptId)`, `scoreCoding(tx, attemptId)`

You use	From
`CodeExecutionService.run` / `submit` (stub returns the example RunResult / SubmitResult)	Majed
`buildAttemptOrder`, `applyQuestionOrder`, `applyOptionOrder`	Majed
`IntegrityService.issueSessionKey`, `getIntegrity`, `AttemptSessionGuard` (stubs from Phase 0)	Esmaeel
Guards, `ClockService`, `round1`, pipe, filter	Esmaeel

5 Rules for You

- Only edit your own files: `attempt-state/`, `student/`.
- Do not run, evaluate, or inspect student code yourself (only through `CodeExecutionService`).
- Do not send `isCorrect` or hidden cases in any student response.
- Do not store `0` for an expired attempt.
- Do not use `new Date()` for "now" (use `ClockService`). Do not use `10`, `20`, or `10000` outside `constants.ts`.
- Do not write instructor content, results, or auth code. Do not write frontend code.

6 Presentation Part

Your part in the English presentation: **backend-owned timing, grace, expiry, `null` vs `0`, and the finish transaction**. Prepare your part and one rule from another member's area, and be ready to explain your own code.